Hacettepe University ELE432 Advanced Digital Design 03/2026

Assignment 1: Warm-Up – Traffic Light Controller FSM (with Delay)

Overview: In class, we discussed the basic, immediate-transition version of the Traffic Light Controller. For this warm-up assignment, you will step it up by implementing the **timed version** of this Finite State Machine (FSM) in SystemVerilog, which includes a 5-second delay for the yellow lights.

Modern hardware development relies heavily on version control, so a major component of this assignment is also setting up your Git workflow and submitting via our homework portal.

Phase 1: Version Control Setup (Git)

Before writing any code, you must set up a Git repository to track your work individually.

1. **Create an Account:** If you do not already have one, create a free individual account on GitHub (or GitLab/Bitbucket).
2. **Create a Repository:** Create a new repository named `Traffic-Light-FSM`.
3. **Initialize Git:** Clone this repository to your local machine. You will use this repository to commit and push your SystemVerilog files as you write and test your code.

Phase 2: SystemVerilog Implementation

Design and write the SystemVerilog code for the Traffic Light Controller FSM as described in the state diagram.

System Inputs & Outputs:

- **Inputs:** `clk`, `reset`, and `TAORB`.
  - *Note on TAORB:* `TAORB` is an input (e.g., from a keypad) indicating that traffic at Street A is present while traffic at Street B is empty. `~TAORB` indicates the opposite.
- **Outputs:** Light states for Street A and Street B (Green, Yellow, Red).

FSM Behavior & Timer Rules: You must implement a 4-state FSM (`S0` to `S3`) with an internal timer/counter to manage the yellow light delays.

- **State 0 (S0):** LA is **Green**, LB is **Red**. The system stays here while `TAORB` is true. When `~TAORB` becomes true, transition to S1.
- **State 1 (S1):** LA is **Yellow**, LB is **Red**. The system must hold this state for 5 time units (e.g., 5 seconds/clock cycles). It loops while `~TAORB` is true AND `TIMER < 5`. Once `TIMER = 5`, transition to S2.

- **State 2 (S2):** LA is **Red**, LB is **Green**. The system stays here while `~TAORB` is true. When `TAORB` becomes true, transition to S3.
- **State 3 (S3):** LA is **Red**, LB is **Yellow**. Similar to S1, the system must hold this state for 5 time units. It loops while `TAORB` is true AND `TIMER < 5`. Once `TIMER = 5`, transition back to S0.

Implementation Requirements:

- Use standard, synthesizable FSM design practices thought in class (separating your state register, next-state logic, and output logic).
- Implement an internal counter that increments during states S1 and S3, and resets appropriately when leaving those states.
- Commit your completed `.sv` files to your Git repository and push them to your remote server.

Phase 3: Submission Instructions

Your final submission will be handled through the **star** portal. We do not accept direct file uploads; you must submit your Git repository link.

1. Verify that all your final code is pushed to your remote Git repository.
2. Copy the URL of your repository. *(Note: If your repository is private, ensure you have invited the instructor/TAs as collaborators so we can view your code).*
3. Log in to the **star** homework portal.
4. Navigate to the "Warm-Up FSM" assignment.
5. Post your repository URL.